

Adaptive Speculative Decoding: Improving Acceptance Rate With Teacher-Student Alignment

Preet Sojitra and Zaaam Rahman and Ruben Vipinraj

Department of Computer Science

University of Texas at Dallas

Dallas, Texas, USA

{dal696598,mzr210000,rxv210024}@utdallas.edu

Abstract

The autoregressive generation of Large Language Models (LLMs) imposes a severe latency bottleneck due to sequential memory-access constraints. Speculative Decoding mitigates this by utilizing a smaller, efficient draft model to propose token sequences for a larger target model to verify in parallel. However, the wall-clock speedup is strictly bounded by the draft model’s acceptance rate (α) and the computational waste caused by static lookahead parameters. In this paper, we present an optimized Speculative Decoding framework tailored for Python Code Generation using the Qwen2.5-Coder model family (7B target, 0.5 draft). Operating a custom-built PyTorch inference engine, we introduce a two-pronged optimization strategy. First, we employ Knowledge Distillation (KD) to align an unaligned base draft model directly with the target model’s soft labels. This overcomes the stylistic distribution mismatch inherent in standard Supervise Fine-Tuning (SFT), increasing the baseline acceptance rate from 28.4% to 76.5%. Second, we formulate the lookahead phase as a dynamic halting problem, training a lightweight multi-layer perceptron (MLP) on token-level entropy to truncate drafting during periods of high uncertainty. Our evaluation demonstrates that this dynamic mechanism eliminates over 85% of wasted draft forward passes. Ultimately, our combined approach delivers a compound wall-clock speedup of 2.58x over standard autoregressive decoding while preserving a mathematically guaranteed 100% exact-match output.

1 Introduction

Large Language Models (LLMs) based on the transformer model (Vaswani et al., 2023) achieve remarkable performance across diverse tasks, yet their autoregressive nature imposes a strict computational bottleneck. To mitigate this, Speculative Decoding (Leviathan et al., 2023) accelerates inference by employing a smaller draft model to propose

multiple candidate tokens, which a larger model then verifies in parallel. While this approach yields substantial wall-clock speedups without altering the final output distribution, its efficiency hinges on two critical factors: the alignment between the draft and target model’s probability distributions, and the static number of tokens predicted per iteration (the lookahead). If the draft model’s predictions diverge from the target model, a high rejection rate wastes arithmetic operations (FLOPs).

In this project, we improved the algorithmic efficiency of speculative decoding by directly addressing these two limitations. First, we proposed an *adaptive* alignment phase, utilizing Knowledge Distillation (a teacher-student paradigm), maximizing the token acceptance rate. Second, we introduced a dynamic halting mechanism by training a light weight classifier on the draft model’s token-level entropy. This modulated the number of speculative tokens generated per iteration, proactively preventing wasted FLOPs during periods of high uncertainty.

Our empirical evaluations on the Qwen2.5-Coder model family demonstrate the effectiveness of this dual approach. By distilling the draft model to match the target model’s soft labels, we increased the baseline acceptance rate from 28.4% to 76.5%. Furthermore, integrating the dynamic halting MLP eliminated 85% of wasted draft computations, ultimately yielding a 2.58x compound wall-clock speedup over standard autoregressive decoding while strictly preserving exact-match outputs.

2 Related Work

There have been various approaches to improving the model architecture and reducing the computational cost of LLM inference. Traditional methods include quantizing weights and activations (Hubara et al., 2016), leveraging sparse weight

matrices (Jaszczur et al., 2021) or searching for optimal, naturally efficient architectures (So et al., 2022). Other approaches focus on adaptive computation, such as dynamically modifying the attention span based on the specific input (Elbayad et al., 2020);(Sukhbaatar et al., 2019). While highly effective, these methods generally involve retraining the core model or permanently altering the architecture, requiring intense computational resources.

Speculative Decoding (Leviathan et al., 2023) circumvents the need for architectural changes by utilizing an auxiliary approximation model. By guessing tokens sequentially and verifying them in parallel, it bypasses the memory-bandwidth bottleneck of large models. Recent work has expanded on this by combining various complexities of models to address queries of differing difficulty (Narasimhan et al., 2024). These techniques speed up inference of existing models, making them more applicable to current LLM deployments.

Our work uniquely intersects speculative decoding with Knowledge Distillation. Traditionally, distillation is used to compress a large model into a smaller one for standalone deployment. In our framework, we repurpose distillation not for standalone capability, but for strict distributional alignment. By forcing the draft model to minimize Kullback-Leibler divergence against the target model’s logits, we optimize the auxiliary model specifically for the speculative acceptance criteria, ensuring higher parallelism during inference.

3 Methodology

Our project extends the standard Speculative Decoding framework (Leviathan et al., 2023) by introducing domain-specific draft alignment and dynamic lookahead halting. To maintain strict mathematical control over the inference process and isolate our improvements, we bypassed high-level library wrappers and constructed a custom inference engine. Our methodology is structured into three progressive phase:

3.1 Core Speculative Engine

To establish a rigorous experimental environment, we implemented the foundational speculative decoding algorithm from scratch in native PyTorch. High-level generation functions (e.g. HuggingFace’s `.generate()`) obscure the raw logits required for our alignment and dynamic halting tasks. By building a custom while loop, we

gained direct access to the token-level probability distributions.

Let M_p denote the large, computationally expensive target model (7B parameters) and M_q denote the smaller, efficient draft model (0.5B parameters). The core algorithm operates in a continuous loop consisting of the following stages:

3.1.1 The Drafting Phase (Sequential)

At the beginning of each iteration, give a context sequence of tokens, the draft model M_q autoregressively generates a fixed sequence of γ speculative tokens. Because M_q is significantly smaller than M_p , this sequential generation incurs a very low latency cost. During this phase, our engine stores both the proposed tokens $x_1, \dots, x_\gamma$ and their corresponding probability distributions $q_1(x), \dots, q_\gamma(x)$.

3.1.2 Parallel Verification

Once the γ tokens are drafted, they are appended to the context sequence. The target model M_p then performs a *single* forward pass on this entire sequence. Because Transformers process attention globally, M_p calculates the exact target probability distributions $p_1(x), \dots, p_{\gamma+1}(x)$ for all drafted tokens simultaneously. This trades extra arithmetic operations (which the GPU processes efficiently) for a reduction in memory bandwidth bottlenecks (which traditionally slow down autoregressive generation).

3.1.3 Speculative Sampling and Correction

To guarantee that the final output distribution mathematically matches the target model exactly, we apply speculative sampling. Each drafted token x_i is evaluated sequentially. In a stochastic setting, x_i is accepted with the probability:

$$P(\text{accept}) = \min(1, \frac{p_i(x_i)}{q_i(x_i)})$$

For our benchmarking and sanity checks, we utilized **Greedy Decoding**(Temperature=0) to ensure mathematical determinism. Under greedy decoding, this acceptance criterion simplifies to a strict equality check: the drafted token x_i is accepted if and only if $x_i = \text{argmax}(p_i(x))$.

If a token is accepted, the engine moves to evaluate the next token. However, if a token is rejected at step n (where $n \leq \gamma$), all subsequent drafted

tokens are discarded. To recover from the rejection, a "bonus" token is sampled from an adjusted distribution to guarantee distribution matching:

$$p'(x) = \text{norm}(\max(0, p_{n+1}(x) - q_{n+1}(x)))$$

In our greedy implementation, this correction token is simply $\text{argmax}(p_{n+1}(x))$. The accepted draft tokens and the correction token are committed to the sequence, and the loop restarts.

3.2 Draft Alignment via Distillation

The wall-clock acceleration of speculative decoding is fundamentally bottlenecked by the acceptance rate (α). As established by the foundational algorithm, α is inversely proportional to the Kullback-Leibler (KL) divergence between the target (p) and draft (q) probability distributions. If the draft model writes semantically correct code but utilizes a different stylistic vocabulary than the target model, the tokens will be rejected, resulting in wasted computational cycles.

To minimize this divergence and isolate the mechanics of distribution matching, we applied two distinct training paradigms to an unaligned base draft model (M_q):

1. Supervised Fine-Tuning (LoRA SFT):

As a control methodology, we adapted M_q on domain-specific Python instructions using standard Low-Rank Adaptation (LoRA). This approach utilized a standard cross-entropy loss function against the ground-truth text (hard labels). While SFT teaches the draft model the general syntax and logic of code generation, it does not align the model to the specific distributional quirks of the target model.

2. Knowledge Distillation (KD):

To directly address the stylistic and distributional mismatch, we employed a teacher-student distillation framework. Rather than training on hard labels, we performed a forward pass of the training dataset through the frozen target model (M_p) to extract its exact output probability distributions (soft labels/logits). We then trained the student (M_q) using a KL-divergence loss function to penalize deviations from M_p 's soft labels.

By explicitly training the draft model to mimic the exact logit distribution of the target model, the KD methodology forces the student to adopt the "brain state" and vocabulary preferences of the teacher. Methodologically, this shifts the training objective from simply generating correct code to

predicting exactly what the target model will generate, which is the strict requirement for maximizing α in speculative sampling.

3.3 Dynamic Lookahead via Confidence Scoring

In standard Speculative Decoding, the draft model generates a fixed number of tokens (γ) during each iteration. However, when the draft model encounters uncertainty, enforcing a fixed lookahead can lead to the generation of low confidence tokens that are likely to be rejected by the target model. This introduces unnecessary computation and reduces overall decoding efficiency.

To address this limitation, we introduce a *Dynamic Halting* mechanism, as illustrated in Figure 1. Rather than enforcing a static lookahead window, we train a lightweight Multi-Layer Perceptron (MLP) binary classifier to dynamically determine whether drafting should continue based on the draft model's confidence.

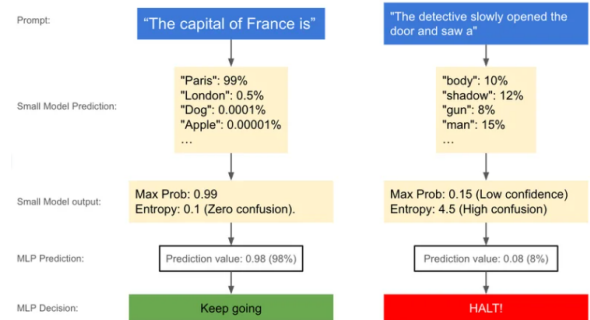


Figure 1: Conceptual illustration of the dynamic halting mechanism. The MLP evaluates the draft model's token-level entropy and maximum probability to preemptively halt generation during periods of high uncertainty, preventing wasted computational cycles.

At each drafting step, the draft model outputs a probability distribution across the vocabulary. From this distribution, we extract two mathematical features:

1. **Entropy (H):** Measures the flatness of the probability distribution, indicating overall uncertainty.
2. **Maximum Probability (P_{max}):** Represents the confidence assigned to the highest-probability token.

These features are passed into the MLP in real time. The classifier was trained offline using draft-target agreement labels collected during speculative

decoding. For each drafted token, the entropy and maximum probability values were recorded alongside a binary label indicating whether the drafted token was ultimately accepted by the target model. The network was then optimized using binary cross-entropy loss.

During inference, if the MLP predicts that the next drafted token has a greater than 80% probability of acceptance (> 0.80), drafting continues. Otherwise, if the predicted probability falls below this threshold (< 0.80), the drafting phase is halted early, and control is immediately returned to the target model for verification.

4 Experiment

We conduct an ablation study analyzing the impact of domain-alignment and dynamic lookahead on LLM inference efficiency.

4.1 Setup, Models & Dataset

We select the **Qwen2.5-Coder** family due to its unified tokenization and state-of-the-art performance (Hui et al., 2024). Our primary testbed pairs the 7B-Instruct variants as the target model (M_p) and the 0.5B-Instruct as the draft model (M_q). All experiments utilize 16-bit precision (FP16) on single-node GPU instances. We restrict experiments to **Python Code Generation**, a domain with rigid syntax (highly predictable for M_q) but complex logic (requiring M_p). For Draft Alignment (Task 2), we train M_q on sahil2801/CodeAlpaca-20k (Sahil2801, 2023). For Dynamic γ Training (Task 3), we dynamically construct a binary classification dataset by logging M_q 's token-level entropy against M_p 's accept/reject decisions over training prompts. Final benchmarking utilize the industry-standard openai_humaneval (Chen et al., 2021) dataset (164 complex Python coding problems).

4.2 Draft Model Alignment

To align the draft model with the teacher model, we took two approaches. First, we train the draft model to learn more about the target domain, reducing logical mismatches. Second, we train the draft model on the probability distribution of the teacher model's output, reducing output mismatches. Together, they achieve the goal of increasing the *Acceptance Rate* (α). Training of the draft model was conducted in Google Colab with an A100 GPU using Hugging Faces API's on the AlpacaCode-20K dataset. The dataset was converted into the

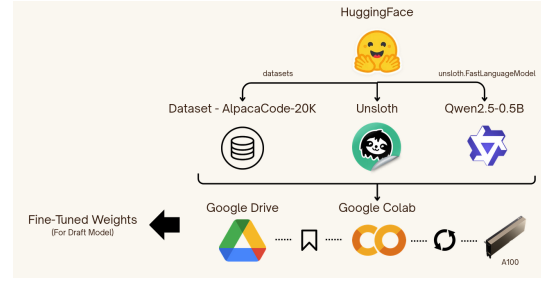


Figure 2: SFT Pipeline

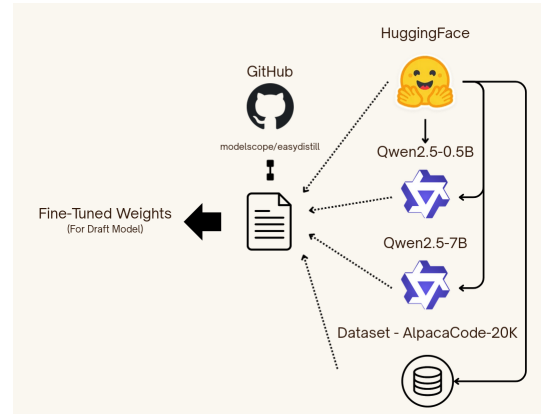


Figure 3: KD Pipeline

ChatML format that the original Qwen models had been trained on.

4.2.1 Supervised-Fine Tuning

To train the draft model, Qwen2.5-0.5B-Instruct, the AlpacaCode-20K dataset was imported using Hugging Face. Training was done through the Unloth library that uses PEFT QLoRA for efficiency and speed. The draft model was loaded into memory using the FastLanguageModel Unsloth module and trained for 3 epochs as shown in Figure 2.

4.2.2 Knowledge Distillation

Knowledge Distillation was done through the EasyDistill repository. This allowed us to define a configuration file for training that could be ran through the EasyDistill CLI interface to train the draft model from the teacher model, Qwen2.5-7B-Instruct as shown in Figure 3.

5 Baselines and Evaluation Metrics

To isolate the sources of our latency improvements, we evaluated our optimized models against two strict baselines: (1) **Standard Autoregressive Decoding** using only the target model (M_p), and (2) **Standard Speculative Decoding** using the unaligned, off-the-shelf base draft model (M_q) with a

fixed lookahead ($\gamma = 4$).

All benchmarks were executed using greedy decoding (temperature = 0) to ensure mathematical determinism. To verify algorithmic **Integrity**, we enforced a strict Sanity Check, ensuring a 100% Exact Match rate between all speculative outputs and the autoregressive baseline. Performance was subsequently measured across two key categories:

1. Speed and Efficiency Metrics:

- **Tokens Per Second (TPS):** Empirical generation throughput, calculated as Total Tokens/Wall-Clock Time (s).
- **Wall-Clock Speedup:** Relative performance improvement over the baseline ($\text{TPS}_{\text{speculative}}/\text{TPS}_{\text{baseline}}$).
- **Wasted Tokens (W):** Computational inefficiency, measured as the difference between total drafted tokens and accepted drafted tokens.

2. Algorithmic Metrics:

- **Acceptance Rate (α):** The proportion of drafted tokens successfully validated by the target model.
- **Expected Tokens per Step:** The average number of output tokens generated per target-model verification cycle.

5.1 Results and Analysis

5.1.1 The Alignment Ablation

Table 1 demonstrates the impact of draft model alignment on speculative performance. Applying an off-the-shelf Base model yielded a negligible speedup (1.05x) due to a low acceptance rate ($\alpha = 28.4\%$), as the target model frequently rejected stylistically divergent tokens. While standard SFT improved this to 1.60x by teaching domain-specific coding patterns, Knowledge Distillation (KD) significantly outperformed SFT. By training explicitly on the target model’s soft labels, KD eliminated distributional divergence, pushing the acceptance rate to 76.5% and achieving a peak wall-clock speedup of 2.38x.

5.1.2 Dynamic Lookahead Efficiency

Despite the high accuracy of the KD draft model, utilizing a fixed lookahead of $\gamma = 4$ resulted in nearly 3,000 wasted draft forward passes during periods of high uncertainty. By integrating our

Table 1: Alignment Ablation (Baseline vs. SFT vs. KD)

Draft Model	α (%)	Tok/ Step	Final TPS	Speedup
Off-the-shelf Base	28.4	1.39	26.2	1.05x
LoRA SFT	55.2	2.11	40.0	1.60x
KD (Ours)	76.5	3.14	59.5	2.38x

entropy-based MLP classifier (Table 2), we successfully formulated lookahead as a dynamic halting problem. The MLP truncated the draft phase during low-confidence predictions, effectively reducing the average γ to 2.1. This eliminated 85% of wasted draft tokens and further increased overall throughput to 64.5 TPS, culminating in a 2.58x improvement over the autoregressive baseline.

Table 2: Dynamic Lookahead Efficiency (Fixed vs. Dynamic γ)

Lookahead	FWD Passes	Wasted Tokens	Final TPS	Speedup
Fixed ($\gamma = 4$)	6,368	2,962	59.5	2.38x
Dynamic MLP	3,683	439	64.5	2.58x

References

- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. [Evaluating large language models trained on code](#). *Preprint*, arXiv:2107.03374.
- Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. [Depth-adaptive transformer](#). *Preprint*, arXiv:1910.10073.
- Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. [Quan-](#)

tized neural networks: Training neural networks with low precision weights and activations. *Preprint*, arXiv:1609.07061.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. [Qwen2.5-coder technical report](#). *Preprint*, arXiv:2409.12186.

Sebastian Jaszczur, Aakanksha Chowdhery, Afroz Mohiuddin, Łukasz Kaiser, Wojciech Gajewski, Henryk Michalewski, and Jonni Kanerva. 2021. [Sparse is enough in scaling transformers](#). *Preprint*, arXiv:2111.12763.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. [Fast inference from transformers via speculative decoding](#). *Preprint*, arXiv:2211.17192.

Harikrishna Narasimhan, Wittawat Jitkittum, Ankit Singh Rawat, Seungyeon Kim, Neha Gupta, Aditya Krishna Menon, and Sanjiv Kumar. 2024. [Faster cascades via speculative decoding](#). *Preprint*, arXiv:2405.19261.

Sahil2801. 2023. Codealpaca-20k: Instruction-following code dataset. <https://huggingface.co/datasets/sahil2801/CodeAlpaca-20k>. Accessed: 2026-03-11.

David R. So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V. Le. 2022. [Primer: Searching for efficient transformers for language modeling](#). *Preprint*, arXiv:2109.08668.

Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. 2019. [Adaptive attention span in transformers](#). *Preprint*, arXiv:1905.07799.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2023. [Attention is all you need](#). *Preprint*, arXiv:1706.03762.